UTN FRM · INGENIERÍA EN SISTEMAS DE INFORMACIÓN · CIENCIA DE DATOS 2026

El sesgo de muestreo por tier

Qué encontramos en el dataset, si afecta al análisis que queremos hacer, y qué opciones tenemos para decidir

Proyecto Integrador · Comisión 5K09 · Grupo 5K09-02

Informe complementario a `documentacion-proyecto.pdf`

6 de septiembre de 2026

Medido sobre **lol_2026-09-06.csv** — 19.145 filas × 137 columnas

Resumen en 30 segundos

LO QUE HAY QUE SABER SI NO SE LEE NADA MÁS

1. Encontramos un problema: las partidas de cada servidor salieron casi todas de jugadores del mismo nivel. NA quedó lleno de plata/bronce, LAN de grandmaster. No fue a propósito.

2. La buena noticia: el análisis que teníamos pensado —ver si el elo predice la duración— **sigue siendo válido**. Lo verificamos: el efecto del elo no desaparece cuando se quita el efecto del servidor.

3. El costo real: el sesgo hace que el elo parezca predecir *menos* de lo que probablemente predice. Nos juega en contra, no a favor. No infla resultados.

4. Lo que hay que decidir: si dejamos el dataset como está declarando el sesgo (barato, defendible), si agregamos una columna para poder controlarlo (una línea de código), o si volvemos a recolectar para emparejar la muestra (caro, y quedan tres días).

Contenido

Parte I · Qué encontramos

- Cómo se consigue una partida
- Dónde se rompe
- La evidencia en los datos
- Por qué KR es la prueba

Parte II · ¿Arruina nuestro análisis?

- La pregunta correcta
- La prueba: sacar el servidor del medio
- Los dos matices
- Lo que el sesgo sí nos cuesta

Parte III · Las opciones

- Opción A · Declararlo y seguir
- Opción B · Agregar la columna de plataforma
- Opción C · Rebalancear la muestra
- Recomendación

Anexo · Los números crudos

PARTE I

Qué encontramos

1. Cómo se consigue una partida

Para entender el problema hay que entender primero cómo llegamos a los datos, porque el problema está justamente ahí.

Riot no permite pedir partidas. No existe un endpoint que diga «dame 20.000 partidas recientes». Lo único que Riot publica es un **ranking de jugadores**, ordenado por nivel: hierro, bronce, plata, oro, platino, esmeralda, diamante, master, grandmaster y challenger.

Entonces el único camino posible es este:

```
elegir un NIVEL de la escalera  (bronce, oro, diamante…)
        │
        ▼
obtener los JUGADORES de ese nivel   ← 205 por página
        │
        ▼
a cada jugador, pedirle SUS ÚLTIMAS PARTIDAS   ← 15 por jugador
        │
        ▼
bajar el detalle de cada partida    → BRONCE
```

Nuestro código elige el nivel de arranque **al azar**, y lo hace a propósito. La idea era buena: si arrancáramos siempre por hierro, toda corrida que se cortara antes de terminar nos dejaría un dataset de puros jugadores malos. Mezclando el orden, en teoría cada corrida toca niveles distintos y la muestra queda pareja.

2. Dónde se rompe

La cuenta que nadie hizo:

> EL NÚMERO QUE ROMPE TODO
>
> Cada nivel tiene **205 jugadores** disponibles por página, y a cada jugador le pedimos **15 partidas**.
>
> **205 × 15 ≈ 3.000 partidas de un solo nivel**
>
> Y la cuota de nuestra corrida era de **3.334 partidas por servidor**. Prácticamente lo mismo.

El resultado es que **la corrida llena su cupo dentro del primer nivel que le tocó, y termina antes de pasar al segundo**. El orden mezclado que tanto nos preocupaba nunca llega a recorrerse: se mezcla una lista que se abandona después del primer elemento.

Una analogía: es como querer encuestar a todo el país, agarrar un barrio al azar, y descubrir que con ese solo barrio ya completaste el cupo de encuestas. Que el barrio sea distinto cada vez no arregla nada — seguís teniendo un solo barrio por encuesta.

> LA CONCLUSIÓN PRECISA
>
> Mezclar el orden **no elimina el sesgo: lo vuelve aleatorio en vez de predecible**. Antes sabíamos que íbamos a tener puro bronce. Ahora no sabemos qué vamos a tener, pero seguimos teniendo un solo nivel por servidor. En algunos sentidos es peor, porque es más difícil de notar.

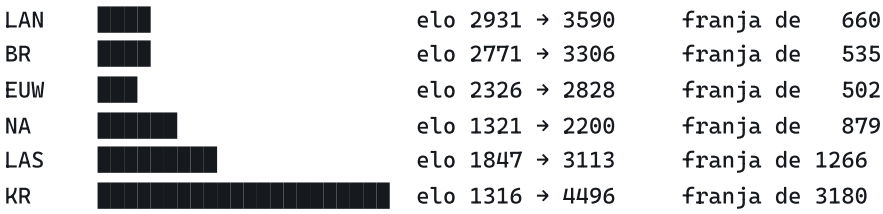
3. La evidencia en los datos

Esto no es teoría. Así quedó repartido el nivel de los jugadores en las 19.145 filas del dataset:

SERVIDOR	PARTIDAS	NIVELES DOMINANTES	ELO MEDIO
`la1` (LAN)	3.355	GRANDMASTER 39% · MASTER 23% · DIAMOND 21%	3.231
`br1` (BR)	3.356	DIAMOND 49% · GRANDMASTER 22% · MASTER 14%	3.036
`euw1` (EUW)	2.950	EMERALD 48% · DIAMOND 33%	2.585
`kr` (KR)	3.430	SILVER 29% · BRONZE 26% · **CHALLENGER 25%**	2.254
`la2` (LAS)	3.066	GOLD 37% · PLATINUM 32%	2.178
`na1` (NA)	2.988	SILVER 33% · BRONZE 29%	1.627

El elo medio va de **1.627 en NA a 3.231 en LAN**: el doble. Eso no refleja ninguna diferencia real entre servidores — refleja en qué escalón de la escalera arrancó cada corrida.

Y no es que dentro de cada servidor haya variedad. Mirando dónde cae el 80% central de las partidas:



Cinco de los seis servidores tienen a casi todas sus partidas apretadas en una franja angosta de nivel. Sólo KR está bien repartido — y por una razón muy puntual.

4. Por qué KR es la prueba

KR es el único servidor que quedó con jugadores de todos los niveles mezclados, y eso **confirma el mecanismo** en vez de contradecirlo.

Los registros de la corrida muestran exactamente por dónde entró cada tarea a la escalera:

SERVIDOR	PRIMER INTENTO (5/9)	SEGUNDO INTENTO, TRAS LA HIBERNACIÓN (6/9)
KR	`BRONZE I`, `BRONZE II`	`CHALLENGER I`
NA	`BRONZE I`, `BRONZE II`	`PLATINUM I`
EUW	`EMERALD I`	`PLATINUM I`
LAS	`GOLD I`	`MASTER I`

Una o dos páginas de escalera por intento. Nada más. Y KR tuvo la casualidad de entrar por **los dos extremos opuestos**: bronce la primera vez, challenger la segunda. Los dos intentos quedaron acumulados en el mismo servidor, y por eso es el único con dos «jorobas».

> UN ACCIDENTE QUE RESULTÓ ÚTIL
>
> KR terminó siendo, sin querer, la única muestra bien repartida que tenemos. Como se verá en la Parte II, es el servidor donde mejor se puede medir el efecto del elo — justamente porque es el único donde el elo varía de verdad. La hibernación que nos costó horas nos dejó, de rebote, el mejor dato del dataset.

PARTE II

¿Arruina nuestro análisis?

1. La pregunta correcta

La pregunta del proyecto es si se puede predecir la duración de una partida a partir de las características de los jugadores, los campeones y las condiciones iniciales. **No tenemos pensado analizar si el servidor influye en la duración.** Queremos mirar el nivel de los jugadores.

Entonces la pregunta concreta es: *si no usamos el servidor como variable, ¿el sesgo nos afecta igual?*

La respuesta corta es **sí, el análisis sigue siendo válido**, pero por una razón que hay que poder explicar — y con dos matices que conviene conocer antes de la defensa.

2. La prueba: sacar el servidor del medio

El riesgo que había que descartar era éste: que cuando creemos estar midiendo «el elo predice la duración», en realidad estemos midiendo «los servidores son distintos entre sí». Como cada servidor tiene su propio nivel de elo, las dos cosas están pegadas y podrían confundirse.

La forma de saberlo es **quitarle a cada partida el promedio de su servidor** y ver si el efecto del elo sobrevive. Si el efecto era en realidad del servidor, al hacer esto se desvanece. Si es del elo, se mantiene.

MEDICIÓN	CORRELACIÓN ELO ↔ DURACIÓN	QUÉ SIGNIFICA
Cruda, como está el dataset	**−0,161**	Mezcla el efecto del elo con las diferencias entre servidores.
Quitando el efecto del servidor	**−0,132**	El efecto limpio del elo. **No desaparece.**
Sólo entre servidores (6 promedios)	−0,682	Fuerte, pero son 6 puntos: no es evidencia de nada por sí solo.

De −0,161 a −0,132. **El efecto del elo se mantiene casi entero.** No era un espejismo del servidor.

Y hay una segunda confirmación, más fuerte todavía: el efecto va en **la misma dirección en los seis servidores**, sin una sola excepción. Más elo, partidas más cortas. Un artefacto no se comporta así de consistente.

> CONCLUSIÓN DE LA PARTE II
>
> El plan de analizar el elo en lugar del servidor **es correcto y los datos lo respaldan**. El sesgo no invalida la relación entre nivel de jugador y duración de partida.

3. Los dos matices

MATIZ 1 · EL NÚMERO CRUDO ESTÁ LEVEMENTE INFLADO

Si informamos **−0,161** y decimos «el elo explica la duración», una parte de ese número no viene del elo sino de que estamos comparando servidores distintos entre sí. El número honesto es **−0,132**.

La diferencia es chica, pero existe, y es la clase de detalle sobre la que se pregunta. Conviene informar el número controlado y saber explicar por qué es el correcto.

MATIZ 2 · ESTAMOS SUBESTIMANDO EL ELO, NO SOBREESTIMÁNDOLO

Este es el matiz importante, y es contraintuitivo: **el sesgo nos perjudica, no nos favorece**.

Para medir si el nivel del jugador afecta la duración hace falta tener jugadores de niveles distintos. Si todos son parecidos, no hay nada que medir. Y eso es exactamente lo que pasa en cinco de nuestros seis servidores.

Se ve clarísimo al comparar el efecto del elo servidor por servidor:

SERVIDOR	ANCHO DE LA FRANJA DE ELO	EFECTO DEL ELO
`kr` (KR)	**3.180** — bien repartido	**−0,254**
`la2` (LAS)	1.266	−0,124
`la1` (LAN)	660	−0,089
`euw1` (EUW)	502	−0,077
`br1` (BR)	535	−0,023
`na1` (NA)	879	−0,014

El patrón es transparente: **cuanto más repartido está el elo, más fuerte aparece su efecto**. En KR, el único servidor con jugadores de todos los niveles, el efecto es **−0,254**: casi el doble del promedio controlado. En NA, donde son todos parecidos, es prácticamente cero.

> CÓMO SE DICE ESTO EN LA DEFENSA
>
> *«El efecto del elo aparece donde hay variación de elo y desaparece donde no la hay. Eso es la firma de nuestro sesgo de recolección, no una propiedad del juego. Nuestro dataset subestima el poder predictivo del nivel de jugador; con una muestra pareja, esperaríamos un efecto más cercano al que medimos en KR.»*

4. Lo que el sesgo sí nos cuesta

Ordenado de mayor a menor importancia:

1. **El modelo va a rendir peor de lo que podría.** Con poca variación de elo hay poca señal de la que aprender. Es el costo principal y es real, pero se traduce en un modelo modesto, no en un modelo falso.
2. **No podemos afirmar nada sobre servidores.** Si alguien mira los datos y concluye que «en NA las partidas duran más que en BR» (y el número dice eso: 1.826 s contra 1.687 s), estaría equivocado: no es el servidor, es que NA quedó lleno de jugadores de nivel bajo. Como no pensábamos analizar el servidor, no nos afecta — **pero hay que saber responderlo si lo preguntan**.

3. **La extrapolación es frágil.** El modelo va a conocer bien seis «islas» de elo y poco del terreno intermedio. Predecir sobre un nivel poco representado es menos confiable.

PARTE III

Las opciones

Tres caminos posibles. No son excluyentes: B se puede hacer además de A.

Opción A · Declarar el sesgo y seguir

Ya está hecho

Qué implica: dejar el dataset exactamente como está y documentar el sesgo con sus números, sus causas y sus consecuencias. Ya quedó escrito en la limitación 3 del documento integral, en el banco de preguntas de la defensa y en el docstring de `seeds.py`.

Costo: ninguno, ya está.

A favor: es lo que la materia espera. Detectar, medir y declarar un sesgo propio es mejor nota que no tenerlo, porque demuestra criterio. El dataset cumple todos los requisitos de la Entrega 1: 19.145 filas, clave única, cero nulos, 137 columnas.

En contra: el modelo de la Unidad 3 va a rendir por debajo de su potencial.

Opción B · Agregar la columna de plataforma

Recomendada, y cuesta una línea

El problema concreto: el dataset **no tiene columna de servidor**. Sin ella no se puede controlar el efecto que describimos en la Parte II — ni siquiera para neutralizarlo.

La solución es que el dato ya está adentro del `match_id`, que trae el servidor como prefijo (`KR_8364526977`, `NA1_5635719422`):

```
df["plataforma"] = df["match_id"].str.split("_").str[0]
```

Costo: una línea en la Unidad 2, sin tocar el pipeline. O una columna nueva en `transform.py` si se quiere que salga del CSV directamente — eso sí requiere decidirlo y reprocesar la capa plata (unos 13 minutos, sin gastar API).

La distinción que hay que tener clara: agregar esta columna **no es analizar el servidor**. Es lo contrario — es neutralizarlo para que no ensucie el efecto del elo. Seguimos analizando el nivel de los jugadores; la plataforma entra sólo como control.

Opción C · Rebalancear la muestra

No para la Entrega 1

Qué implica: lanzar corridas nuevas apuntadas a los niveles que faltan en cada servidor, hasta emparejar la distribución.

Costo: alto. Al ritmo medido de ~1.150 partidas/hora, emparejar seis servidores son varias jornadas de descarga, con keys que vencen cada 24 horas y una máquina que no puede suspenderse. Faltan tres días para la entrega.

Además: el código actual no permite elegir el nivel de arranque, así que primero habría que modificar `iter_puuids` para rotar de tier — un cambio de lógica en el corazón de la recolección, a tres días de la defensa.

Cuándo sí: tiene todo el sentido antes de la **Unidad 3 (modelado, 14/10)**, que es cuando el sesgo empieza a costar de verdad. Hay más de un mes, y el bronce ya bajado se conserva: sumar niveles faltantes no obliga a rehacer nada.

Recomendación

LO QUE HARÍAMOS

Para el miércoles: A + B. El sesgo ya está documentado (A) y agregar la columna de plataforma cuesta una línea (B). Con eso el dataset queda entregable, el sesgo declarado, y el análisis del elo se puede hacer bien desde la Unidad 2.

Para octubre: evaluar C. Antes del modelado conviene emparejar la muestra, con tiempo y con `iter_puuids` arreglado. Ahí es donde el sesgo realmente cuesta puntos, y ahí hay margen para hacerlo bien.

Las preguntas que pueden hacer, con la respuesta lista

¿Su muestra es representativa?

No, y lo sabemos medido. Es una muestra de conveniencia sesgada por nivel de jugador: cada servidor quedó con partidas de un rango casi puro, con elo medio de 1.627 a 3.231. La causa es que una página de escalera da 205 jugadores y eso alcanza para llenar la cuota entera de una corrida, así que nunca se pasa al segundo nivel. Se corrige rotando de tier durante la descarga.

Entonces sus conclusiones sobre el elo no valen.

Sí valen, y lo verificamos. Si el efecto del elo fuera en realidad un efecto del servidor, al controlar por servidor tendría que desaparecer. Pasa de −0,161 a −0,132: se mantiene. Y va en la misma dirección en los seis servidores. Lo que sí hace el sesgo es *subestimar* el efecto, no inventarlo: en KR, el único servidor con niveles mezclados, el efecto llega a −0,254.

¿Por qué en NA las partidas duran más que en BR?

No duran más por ser NA. Es que NA quedó lleno de jugadores de plata y bronce, y BR de diamante. Estamos viendo el efecto del nivel de los jugadores, no del servidor. Es un ejemplo directo de la confusión que introduce nuestro sesgo de muestreo, y por eso no sacamos conclusiones por región.

ANEXO

Los números crudos

Todo lo de este informe sale de `include/output/lol_2026-09-06.csv` (19.145 filas × 137 columnas) y de los registros de la corrida del 5–6/9/2026. El elo de una partida es el promedio de las diez columnas `*_player_elo`.

ELO Y DURACIÓN POR SERVIDOR

SERVIDOR	PARTIDAS	ELO MEDIO	DESVÍO DEL ELO	DURACIÓN MEDIA	EFECTO DEL ELO
`br1`	3.333	3.036	262	1.687 s	−0,023
`euw1`	2.950	2.585	223	1.743 s	−0,077
`kr`	3.430	2.254	**1.293**	1.689 s	**−0,254**
`la1`	3.355	3.231	293	1.722 s	−0,089
`la2`	3.066	2.178	440	1.837 s	−0,124
`na1`	3.011	1.627	342	1.826 s	−0,014

> **POR QUÉ LOS CONTEOS NO COINCIDEN CON LA PARTE I**
>
> En esta tabla el servidor se deduce del prefijo del `match_id`; en la Parte I se cuentan los archivos de cada carpeta `plataforma=`. Difieren en unas pocas decenas porque un jugador semilla puede tener partidas jugadas en otra región, que quedan guardadas bajo la carpeta del servidor por el que se llegó a él. El total es el mismo: 19.145.

CORRELACIONES ENTRE ELO Y DURACIÓN

FORMA DE MEDIRLA	VALOR
Cruda, sobre las 19.145 filas	−0,1608
Controlando por servidor (correlación parcial)	**−0,1321**
Entre los promedios de los 6 servidores	−0,6824

CONCENTRACIÓN DEL ELO: EL 80% CENTRAL DE LAS PARTIDAS

SERVIDOR	PERCENTIL 10	PERCENTIL 90	ANCHO DE LA FRANJA
`kr`	1.316	4.496	**3.180**
`la2`	1.847	3.113	1.266
`na1`	1.321	2.200	879
`la1`	2.931	3.590	660
`br1`	2.771	3.306	535
`euw1`	2.326	2.828	502

CÓMO SE CALCULA EL ELO

Es una construcción nuestra, no un dato de Riot — Riot no publica el MMR. La fórmula está en `include/lol/riot.py`:

```
estimated_elo = base_del_tier + bonus_de_division + (puntos_de_liga × 0,5)

IRON 800 · BRONZE 1100 · SILVER 1400 · GOLD 1700 · PLATINUM 2000
EMERALD 2300 · DIAMOND 2600 · MASTER 2900 · GRANDMASTER 3200 · CHALLENGER 3500
bonus de división:  IV 0 · III 75 · II 150 · I 225
```

Un jugador sin ranking clasificatorio recibe 1.500 por defecto. Esto es relevante para leer las tablas: parte de la concentración de elo en los servidores de nivel bajo viene de ese valor por defecto.
